AWS Infrastructure Automation Project Report

Terraform, Ansible, GitHub Actions, EC2, S3 and CloudWatch

This report explains the AWS infrastructure automation project that I built. I have written it in a simple way so it is easy to understand what was built, why each tool was used, and how the final deployment works. The project is not just a basic EC2 setup; it connects infrastructure provisioning, server configuration, CI/CD automation, remote state management, and monitoring into one working flow.

1. Project aim

The aim of this project was to build a small but complete DevOps workflow. I wanted Terraform to create the infrastructure, Ansible to configure the servers, GitHub Actions to automate the deployment, and CloudWatch to provide basic visibility into the running EC2 instances.

The final result is that when I push a change to GitHub, the workflow runs automatically and updates both EC2 servers without manually logging into them.

2. Tools and services used

Tool / Service	How I used it
Terraform	Created the VPC, subnet, route table, security group, EC2 instances, key pair and CloudWatch alarms.
AWS EC2	Hosted two Amazon Linux servers running Nginx.
AWS S3	Stored the Terraform remote state file so GitHub Actions and my laptop use the same state.
Ansible	Installed and configured Nginx on both EC2 instances using one playbook.
GitHub Actions	Automated the Terraform and Ansible deployment whenever changes were pushed to main.
AWS OIDC	Allowed GitHub Actions to access AWS without storing long-term AWS access keys in GitHub.
CloudWatch	Monitored CPU utilization for both EC2 instances using Terraform-created alarms.

3. Architecture overview

The project follows a simple automation path. I push code to GitHub, GitHub Actions authenticates to AWS, Terraform checks and applies the infrastructure, and then Ansible connects to the EC2 instances to deploy the web page.

```
GitHub push
   ↓
GitHub Actions
   ↓
AWS OIDC authentication
   ↓
Terraform apply
   ↓
EC2 infrastructure
   ↓
Ansible configuration
```

```
   ↓
Nginx web page deployment
   ↓
CloudWatch monitoring
```

4. Infrastructure built with Terraform

Terraform was used as the main infrastructure-as-code tool. I created the AWS network first, then the security group, and then the two EC2 instances. The network contains a custom VPC, a public subnet, an Internet Gateway and a route table association. This made the instances reachable over HTTP and SSH.

The security group allows HTTP traffic on port 80. SSH was restricted instead of being left open permanently. For GitHub Actions, the workflow temporarily adds the runner IP address to the security group before Ansible runs, and removes it after the deployment. This keeps the setup cleaner than opening SSH to the whole internet.

I also used Terraform to create two CloudWatch alarms, one for each EC2 instance. These alarms track CPU utilization and show the monitoring part of the infrastructure clearly.

5. Terraform remote state

At first, the Terraform state was only stored locally on my laptop. That would be a problem for GitHub Actions because the runner would not know about the resources already created in AWS. To fix this, I created an S3 bucket and moved the state to an S3 backend.

After migrating to remote state, both my local Terraform commands and GitHub Actions could read the same state file. This was an important step because it made the CI/CD workflow safe to run without accidentally recreating resources.

6. Server configuration with Ansible

After the EC2 instances were created, I used Ansible to configure them. I created an inventory with two hosts, web1 and web2, and used one playbook to install and start Nginx on both servers.

The web page is generated using an Ansible template. This means the same template is used for both servers, but Ansible fills in the server name differently for each instance. On the first server the page shows web1, and on the second server it shows web2.

7. CI/CD automation with GitHub Actions

The GitHub Actions workflow is the part that connects the whole project together. When a change is pushed to the main branch, the workflow checks out the repository, authenticates to AWS through OIDC, sets up Terraform, runs Terraform apply, captures the EC2 IP addresses, prepares SSH, installs Ansible and runs the Ansible playbook.

The workflow also temporarily allows SSH from the GitHub runner public IP. This is needed because GitHub runners do not have a fixed IP address. After Ansible finishes, the workflow removes that SSH rule again. I included this cleanup step so the security group is not left open.

8. Monitoring with CloudWatch

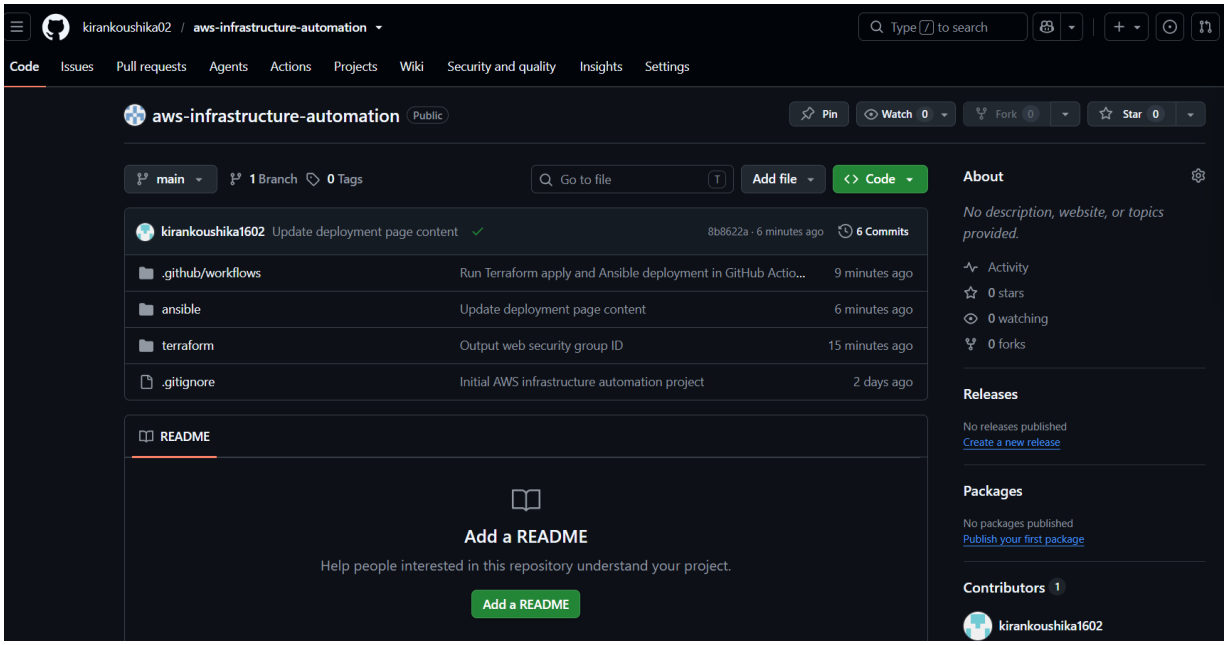
CloudWatch was used to show basic infrastructure monitoring. I created one high CPU alarm for each EC2 instance using Terraform. This gives visibility into the two running servers and supports the monitoring part of the project.

9. Evidence and screenshots

The screenshots below show the project working across GitHub, AWS and the local Terraform environment.

GitHub repository structure

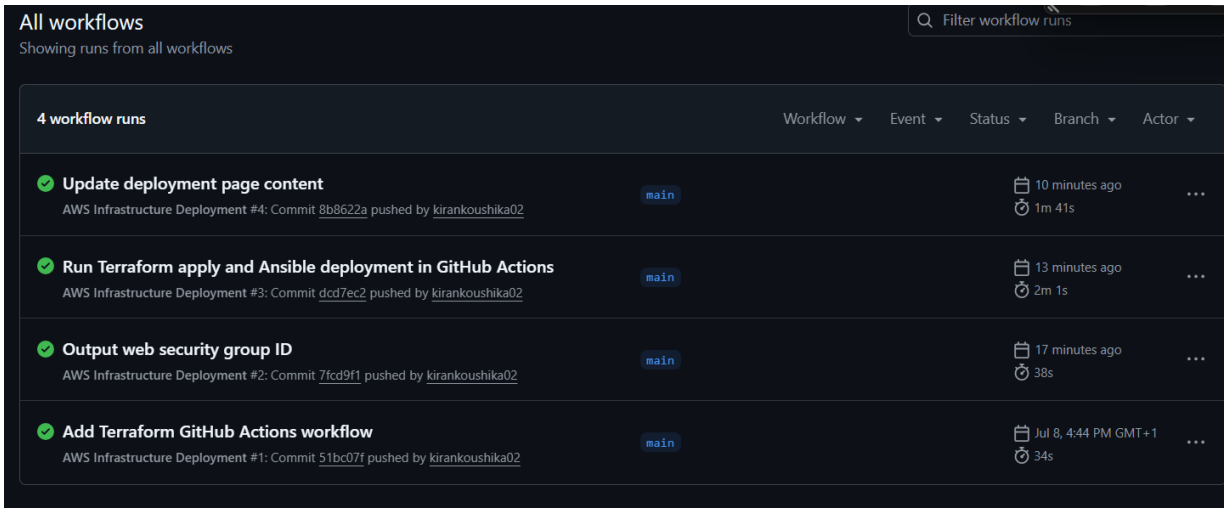
The GitHub repository contains separate folders for Terraform, Ansible, GitHub Actions and screenshots. This helped keep the project clean and easy to follow.



Screenshot: GitHub repository structure

GitHub Actions workflow runs

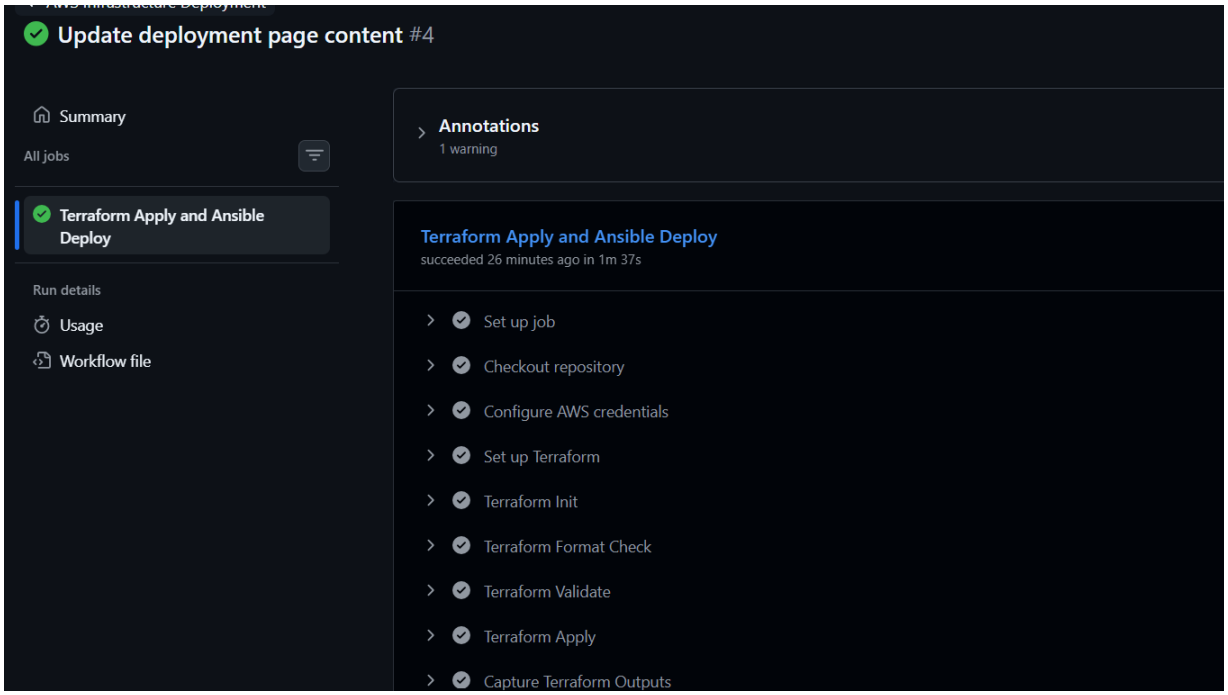
This shows the workflow history after the project was connected to GitHub Actions. Each push triggered a deployment workflow and the final runs completed successfully.



Screenshot: GitHub Actions workflow runs

Successful deployment job steps

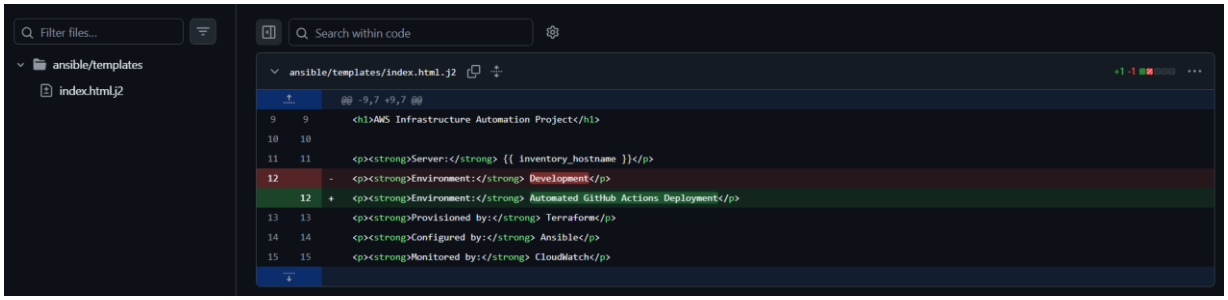
The final GitHub Actions job completed successfully. The important steps were Terraform apply, capturing outputs, allowing temporary SSH access, running Ansible, and removing the temporary SSH rule afterwards.



Screenshot: Successful deployment job steps

Template change that triggered deployment

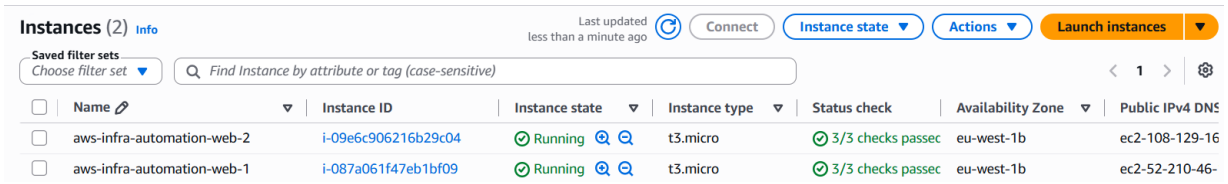
This screenshot shows the actual web page template change that was pushed to GitHub. The successful deployment proved that a code change could update both EC2 servers automatically.



Screenshot: Template change that triggered deployment

Two EC2 instances running

Both EC2 instances are running in the Ireland region. These instances were created by Terraform and later configured by Ansible.



Screenshot: Two EC2 instances running

Web page deployed on web1

The first EC2 instance is serving the custom Nginx page and correctly shows Server: web1.

Screenshot: Web page deployed on web1

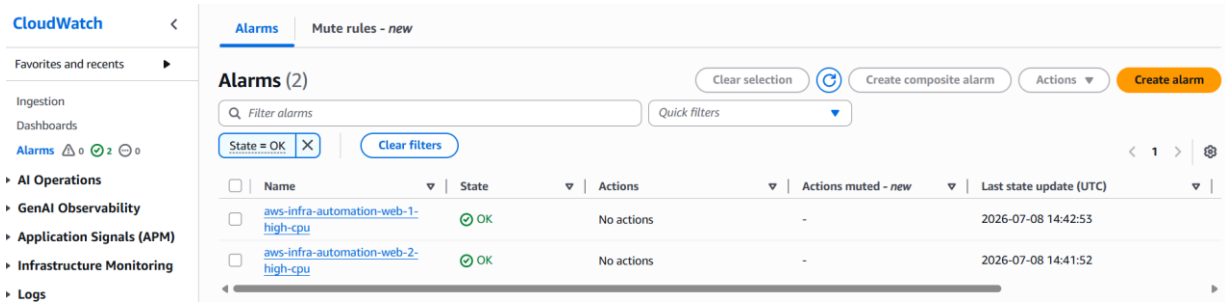
Web page deployed on web2

The second EC2 instance is serving the same template, but Ansible rendered it with Server: web2.

Screenshot: Web page deployed on web2

CloudWatch CPU alarms

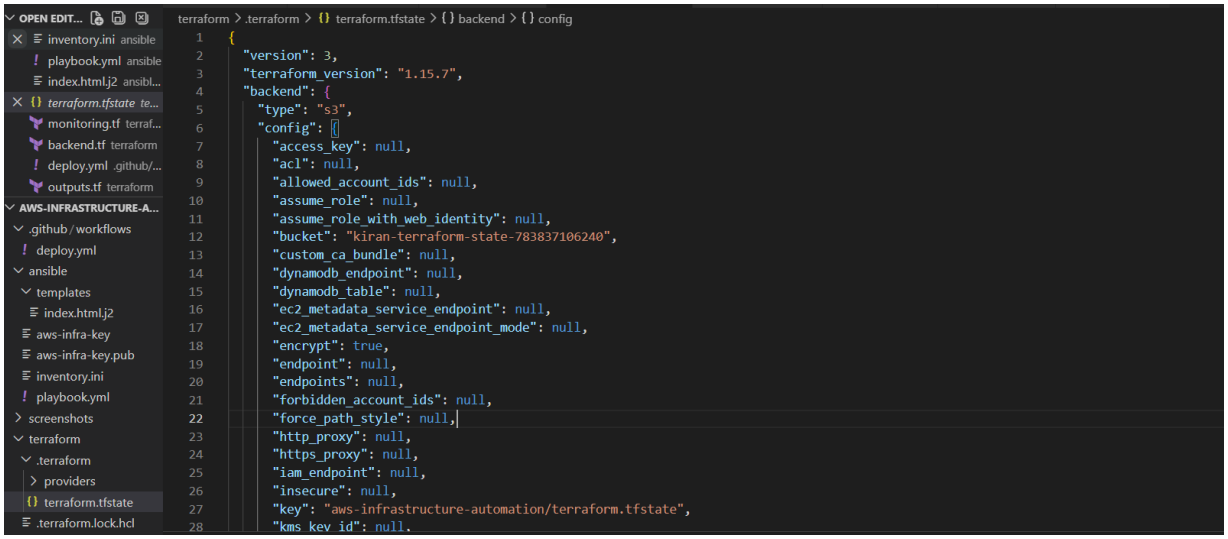
CloudWatch shows one alarm for each EC2 instance. Both alarms were created using Terraform.



Screenshot: CloudWatch CPU alarms

S3 remote Terraform state

The Terraform backend is configured to use S3 remote state. This was needed so GitHub Actions could work from the same state as the local machine.



Screenshot: S3 remote Terraform state

Terraform plan with no changes

The final terraform plan showed no changes, which means the infrastructure in AWS matched the Terraform code and remote state.

```
PS C:\Users\kiran\aws-infrastructure-automation> cd C:\Users\kiran\aws-infrastructure-automation\terraform
PS C:\Users\kiran\aws-infrastructure-automation\terraform> terraform plan
data.aws_ami.amazon_linux: Reading...
aws_key_pair.deployer: Refreshing state... [id=aws-infra-automation-key]
aws_vpc.main: Refreshing state... [id=vpc-0266d26a3d515078d]
data.aws_ami.amazon_linux: Read complete after 0s [id=ami-016c3cb30fb76b122]
aws_internet_gateway.main: Refreshing state... [id=igw-0199177a32af7ee56]
aws_security_group.web: Refreshing state... [id=sg-03858d9723e959647]
aws_subnet.public: Refreshing state... [id=subnet-005b92b6caec2db67]
aws_route_table.public: Refreshing state... [id=rtb-026468edbe9201e5c]
aws_route_table_association.public: Refreshing state... [id=rtbassoc-04b33e4770c24c91e]
aws_instance.web_2: Refreshing state... [id=i-09e6c906216b29c04]
aws_instance.web_1: Refreshing state... [id=i-087a061f47eb1bf09]
aws_cloudwatch_metric_alarm.web_2_high_cpu: Refreshing state... [id=aws-infra-automation-web-2-high-cpu]
aws_cloudwatch_metric_alarm.web_1_high_cpu: Refreshing state... [id=aws-infra-automation-web-1-high-cpu]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are
needed.
```

Screenshot: Terraform plan with no changes

10. Problems I faced and how I fixed them

A few issues came up while building this project. They were useful because they made the project more realistic.

- Terraform worked on Windows PowerShell, but Ansible needed WSL Ubuntu. I separated the workflow clearly: Terraform and Git commands from PowerShell, Ansible from WSL.
- The first Terraform plan failed because AWS CLI credentials were not configured. I fixed this by creating an IAM user for local CLI access instead of using root access keys.
- GitHub Actions needed AWS access, but I did not want to store AWS access keys in GitHub. I fixed this by creating an IAM OIDC provider and a GitHub Actions role.
- GitHub Actions could not SSH into EC2 using my home IP security rule. I fixed this by adding a temporary security group rule for the runner IP during the workflow and removing it afterwards.
- Terraform state had to be moved to S3 so GitHub Actions could safely work with the existing infrastructure instead of treating it like a fresh deployment.

11. What I learned

This project helped me understand how different DevOps tools work together in a real workflow. Terraform was responsible for infrastructure, Ansible handled server configuration, GitHub Actions automated the process, and CloudWatch provided monitoring.

The biggest learning point was state management. Once I moved the Terraform state to S3, the project became more realistic because both my local machine and GitHub Actions were working from the same source of truth.

I also learned why authentication matters in automation. Using GitHub OIDC is better than placing long-term AWS keys in GitHub secrets, and the temporary SSH rule made the deployment more secure than leaving SSH open.

12. Final outcome

The final project is a working automated deployment pipeline. A change to the web page template can be pushed to GitHub, and GitHub Actions will apply Terraform, run Ansible and update both EC2 web servers.

The project is suitable for showing practical experience with AWS infrastructure provisioning, CI/CD automation, configuration management, remote state, and basic cloud monitoring.

13. Resume-ready summary

- Provisioned a custom AWS environment with Terraform, including VPC networking, security groups and two EC2 instances, with S3 remote state management for consistent infrastructure tracking.
- Built a GitHub Actions workflow using AWS OIDC to run Terraform and Ansible automatically, deploying Nginx configuration changes across both EC2 instances.
- Configured CloudWatch CPU alarms for both EC2 instances to provide basic monitoring and visibility into infrastructure health.

14. Cleanup note

The AWS resources should be destroyed when they are no longer needed, especially the EC2 instances, to avoid unnecessary charges. The cleanup command should be run from the Terraform folder:

```
terraform destroy
```